
Categorical and Mixed Type Data

Unit 4

Decision Tree

A decision tree is a type of supervised machine learning algorithm used for both classification and regression tasks.

It works by splitting the dataset into smaller and smaller subsets based on certain conditions, forming a tree-like structure.

The goal is to find the best splits that help separate the data points clearly based on the target variable.

Components of a Decision Tree

- Root Node: The topmost node. It represents the entire dataset, and the first decision (split) is made here.
- Internal/Decision Nodes: These are the points where the dataset is split based on a feature.
- Branches: These represent the outcome of a split (e.g., Yes or No, $\leq$ threshold or $>$ threshold).
- Leaf/Terminal Nodes: These are the final outcomes (classes or predicted values).

How does it work?

The decision tree algorithm chooses the best feature and split point using a criterion

such as:

- Information Gain (used in ID3)
- Gini Impurity (used in CART)

Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



How Splitting Happens in Decision Trees

1. Gini Impurity (Used in CART)

$$\text{Gini} = 1 - \sum_{i=1}^n p_i^2$$

Where p_i is the probability of class i . Gini measures how often a randomly chosen element would be incorrectly labeled.

How Splitting Happens in Decision Trees

2. Entropy (Used in ID3)

$$\text{Entropy} = - \sum_{i=1}^n p_i \log_2 p_i$$

Entropy measures the impurity or randomness in the data.

3. Information Gain

Used to decide the best feature to split on.

$$\text{Information Gain} = \text{Entropy}(\text{Parent}) - \sum \left(\frac{|\text{Child}|}{|\text{Parent}|} \times \text{Entropy}(\text{Child}) \right)$$

4. Mean Squared Error (MSE) — Regression Trees

Used for continuous target variables.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2$$

ID3 (Iterative Dichotomiser 3)

ID3 builds a decision tree top-down, starting from the root and recursively selecting the best attribute to split the data. It uses information gain to decide which attribute to split on.

1. Entropy

Entropy is a measure of impurity or disorder in a dataset. It is calculated as:

$$\text{Entropy}(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

where:

- S is a set of examples,
- p_i is the proportion of examples in class i ,
- c is the number of classes.

ID3 (Iterative Dichotmiser 3)

2. Information Gain

Information Gain measures how much “information” a feature gives us about the class. It is defined as:

$$IG(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

where:

- A is the attribute,
- S_v is the subset of S for which attribute A has value v .

The attribute with the highest information gain is selected to split the node.

ID3 (Iterative Dichotmiser 3)

2. Information Gain

Information Gain measures how much “information” a feature gives us about the class. It is defined as:

$$IG(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

where:

- A is the attribute,
- S_v is the subset of S for which attribute A has value v .

The attribute with the highest information gain is selected to split the node.

ID3 Algorithm

1. Start with the full dataset.
2. Compute entropy of the dataset.
3. For each attribute, compute information gain.
4. Choose the attribute with the highest information gain to split the dataset.
5. Repeat recursively for each subset until:
 - All examples in a subset belong to the same class,
 - There are no more attributes to split on,
 - The subset is empty.

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Outlook

Values (Outlook) = Sunny, Overcast, Rain

$$S = [9+, 5-]$$

$$Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{Sunny} \leftarrow [2+, 3-]$$

$$Entropy(S_{Sunny}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.971$$

$$S_{Overcast} \leftarrow [4+, 0-]$$

$$Entropy(S_{Overcast}) = -\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} = 0$$

$$S_{Rain} \leftarrow [3+, 2-]$$

$$Entropy(S_{Rain}) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.971$$

$$Gain(S, Outlook) = Entropy(S) - \sum_{v \in \{Sunny, Overcast, Rain\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Outlook)$$

$$= Entropy(S) - \frac{5}{14} Entropy(S_{Sunny}) - \frac{4}{14} Entropy(S_{Overcast}) - \frac{5}{14} Entropy(S_{Rain})$$

$$Gain(S, Outlook) = 0.94 - (5/14)*0.971 - (4/14)*0 - (5/14)*0.971 = 0.2464$$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Temp

Values (Temp) = Hot, Mild, Cool

$$S = [9+, 5-]$$

$$Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{Hot} \leftarrow [2+, 2-]$$

$$Entropy(S_{Hot}) = -\frac{2}{4} \log_2 \frac{2}{4} - \frac{2}{4} \log_2 \frac{2}{4} = 1.0$$

$$S_{Mild} \leftarrow [4+, 2-]$$

$$Entropy(S_{Mild}) = -\frac{4}{6} \log_2 \frac{4}{6} - \frac{2}{6} \log_2 \frac{2}{6} = 0.9183$$

$$S_{Cool} \leftarrow [3+, 1-]$$

$$Entropy(S_{Cool}) = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = 0.8113$$

$$Gain(S, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Temp)$$

$$= Entropy(S) - \frac{4}{14} Entropy(S_{Hot}) - \frac{6}{14} Entropy(S_{Mild})$$

$$- \frac{4}{14} Entropy(S_{Cool})$$

$$Gain(S, Temp) = 0.94 - (4/14)*1 - (6/14)*0.9183 - (4/14)*0.8113 = 0.0289$$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Humidity

Values (Humidity) = High, Normal

$$S = [9+, 5-]$$

$$Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{High} \leftarrow [3+, 4-]$$

$$Entropy(S_{High}) = -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} = 0.9852$$

$$S_{Normal} \leftarrow [6+, 1-]$$

$$Entropy(S_{Normal}) = -\frac{6}{7} \log_2 \frac{6}{7} - \frac{1}{7} \log_2 \frac{1}{7} = 0.5916$$

$$Gain(S, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Humidity)$$

$$= Entropy(S) - \frac{7}{14} Entropy(S_{High}) - \frac{7}{14} Entropy(S_{Normal})$$

$$Gain(S, Humidity) = 0.94 - (7/14) * 0.9852 - (7/14) * 0.5916 = 0.1516$$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Wind

Values (Wind) = Strong, Weak

$$S = [9+, 5-]$$

$$Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{Strong} \leftarrow [3+, 3-]$$

$$Entropy(S_{Strong}) = 1.0$$

$$S_{Weak} \leftarrow [6+, 2-]$$

$$Entropy(S_{Weak}) = -\frac{6}{8} \log_2 \frac{6}{8} - \frac{2}{8} \log_2 \frac{2}{8} = 0.8113$$

$$Gain(S, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Wind) = Entropy(S) - \frac{6}{14} Entropy(S_{Strong}) - \frac{8}{14} Entropy(S_{Weak})$$

$$Gain(S, Wind) = Entropy(S) - (6/14)*1 - (8/14)*0.8113 = 0.0478$$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

$$Gain(S, Outlook) = 0.2464$$

$$Gain(S, Temp) = 0.0289$$

$$Gain(S, Humidity) = 0.1516$$

$$Gain(S, Wind) = 0.0478$$

$\{D1, D2, \dots, D14\}$

$[9+, 5-]$

Outlook

Sunny

Overcast

Rain

$\{D1, D2, D8, D9, D11\}$

$\{D3, D7, D12, D13\}$

$\{D4, D5, D6, D10, D14\}$

$[2+, 3-]$

$[4+, 0-]$

$[3+, 2-]$

?

Yes

?

Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Attribute: Temp

Values (Temp) = Hot, Mild, Cool

$$S_{Sunny} = [2+, 3-]$$

$$Entropy(S_{Sunny}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.97$$

$$S_{Hot} \leftarrow [0+, 2-]$$

$$Entropy(S_{Hot}) = 0.0$$

$$S_{Mild} \leftarrow [1+, 1-]$$

$$Entropy(S_{Mild}) = 1.0$$

$$S_{Cool} \leftarrow [1+, 0-]$$

$$Entropy(S_{Cool}) = 0.0$$

$$Gain(S_{Sunny}, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Sunny}, Temp)$$

$$= Entropy(S) - \frac{2}{5} Entropy(S_{Hot}) - \frac{2}{5} Entropy(S_{Mild})$$

$$- \frac{1}{5} Entropy(S_{Cool})$$

$$Gain(S_{Sunny}, Temp) = 0.97 - \left(\frac{2}{5}\right) * 0 - \left(\frac{2}{5}\right) * 1 - \left(\frac{1}{5}\right) * 0 = 0.570$$

Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Attribute: Humidity

Values (Humidity) = High, Normal

$$S_{Sunny} = [2+, 3-]$$

$$Entropy(S) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.97$$

$$S_{high} \leftarrow [0+, 3-]$$

$$Entropy(S_{High}) = 0.0$$

$$S_{Normal} \leftarrow [2+, 0-]$$

$$Entropy(S_{Normal}) = 0.0$$

$$Gain(S_{Sunny}, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Sunny}, Humidity) = Entropy(S) - \frac{3}{5} Entropy(S_{High}) - \frac{2}{5} Entropy(S_{Normal})$$

$$Gain(S_{sunny}, Humidity) = 0.97 - (3/5) * 0 - (2/5) * 0 = 0.97$$

Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Attribute: Wind

Values (Wind) = Strong, Weak

$$S_{Sunny} = [2+, 3-]$$

$$Entropy(S) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.97$$

$$S_{Strong} \leftarrow [1+, 1-]$$

$$Entropy(S_{Strong}) = 1.0$$

$$S_{Weak} \leftarrow [1+, 2-]$$

$$Entropy(S_{Weak}) = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} = 0.9183$$

$$Gain(S_{Sunny}, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Sunny}, Wind) = Entropy(S) - \frac{2}{5} Entropy(S_{Strong}) - \frac{3}{5} Entropy(S_{Weak})$$

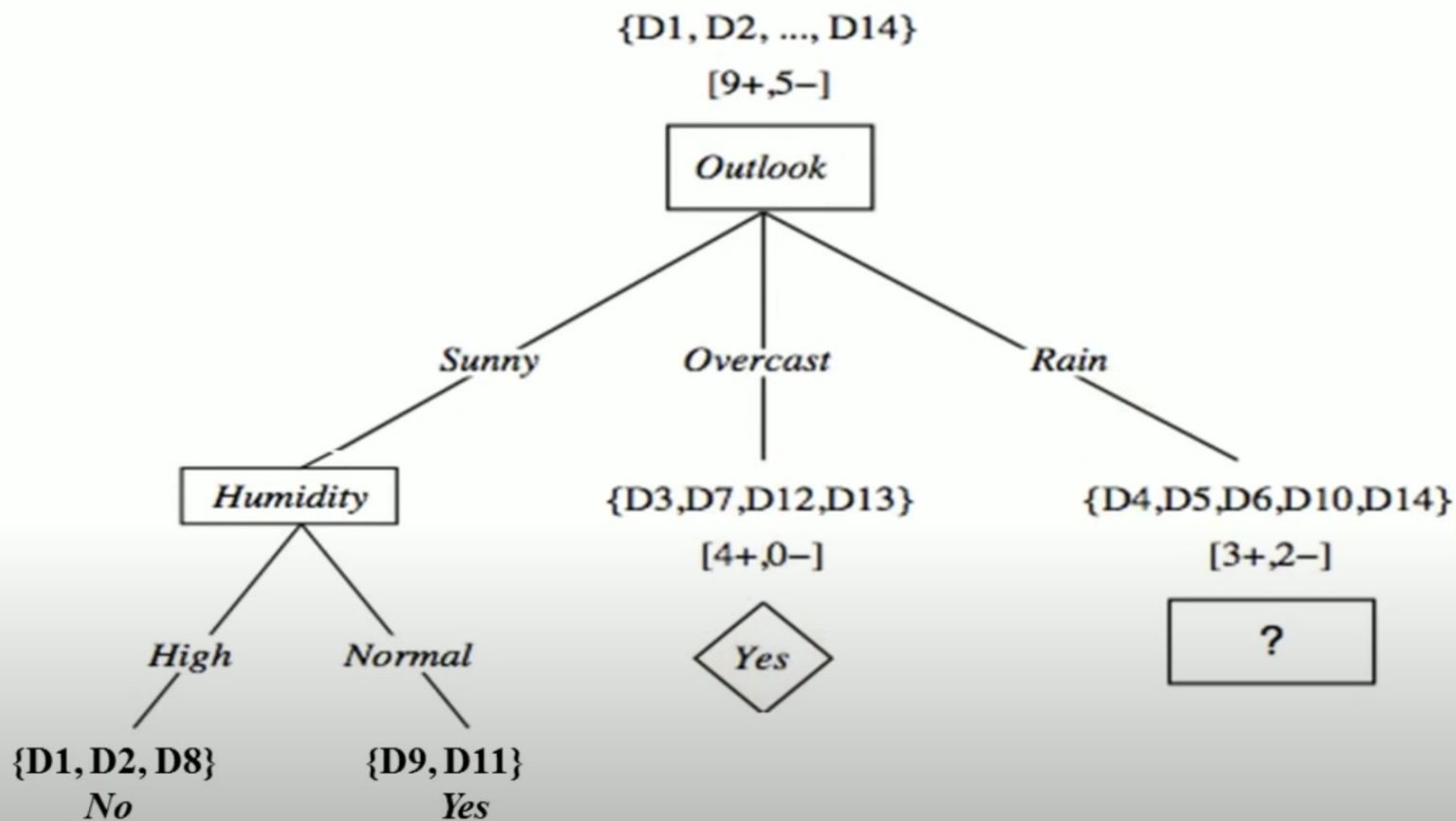
$$Gain(S_{Sunny}, Wind) = 0.97 - (\frac{2}{5}) * 1 - (\frac{3}{5}) * 0.9183 = 0.0192$$

Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

$$Gain(S_{sunny}, Temp) = 0.570$$

$$Gain(S_{sunny}, Humidity) = 0.97$$

$$Gain(S_{sunny}, Wind) = 0.0192$$



Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Attribute: Temp

Values (Temp) = Hot, Mild, Cool

$$S_{Rain} = [3+, 2-]$$

$$Entropy(S_{Sunny}) = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} = 0.97$$

$$S_{Hot} \leftarrow [0+, 0-]$$

$$Entropy(S_{Hot}) = 0.0$$

$$S_{Mild} \leftarrow [2+, 1-]$$

$$Entropy(S_{Mild}) = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.9183$$

$$S_{Cool} \leftarrow [1+, 1-]$$

$$Entropy(S_{Cool}) = 1.0$$

$$Gain(S_{Rain}, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Rain}, Temp)$$

$$= Entropy(S) - \frac{0}{5} Entropy(S_{Hot}) - \frac{3}{5} Entropy(S_{Mild})$$

$$- \frac{2}{5} Entropy(S_{Cool})$$

$$Gain(S_{RAIN}, Temp) = 0.97 - (0/5)*0 - (3/5)*0.9183 - (2/5)*1 = 0.0192$$

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Attribute: Humidity

Values (Humidity) = High, Normal

$$S_{Rain} = [3+, 2-]$$

$$Entropy(S_{Sunny}) = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} = 0.97$$

$$S_{High} \leftarrow [1+, 1-]$$

$$Entropy(S_{High}) = 1.0$$

$$S_{Normal} \leftarrow [2+, 1-]$$

$$Entropy(S_{Normal}) = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.9183$$

$$Gain(S_{Rain}, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Rain}, Humidity) = Entropy(S) - \frac{2}{5} Entropy(S_{High}) - \frac{3}{5} Entropy(S_{Normal})$$

$$Gain(S_{RAIN}, Humidity) = 0.97 - (\frac{2}{5}) * 1 - (\frac{3}{5}) * 0.9183 = 0.0192$$

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Attribute: Wind

Values (wind) = Strong, Weak

$$S_{Rain} = [3+, 2-]$$

$$Entropy(S_{Sunny}) = -\frac{3}{5}\log_2 \frac{3}{5} - \frac{2}{5}\log_2 \frac{2}{5} = 0.97$$

$$S_{Strong} \leftarrow [0+, 2-]$$

$$Entropy(S_{Strong}) = 0.0$$

$$S_{Weak} \leftarrow [3+, 0-]$$

$$Entropy(S_{Weak}) = 0.0$$

$$Gain(S_{Rain}, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Rain}, Wind) = Entropy(S) - \frac{2}{5} Entropy(S_{Strong}) - \frac{3}{5} Entropy(S_{Weak})$$

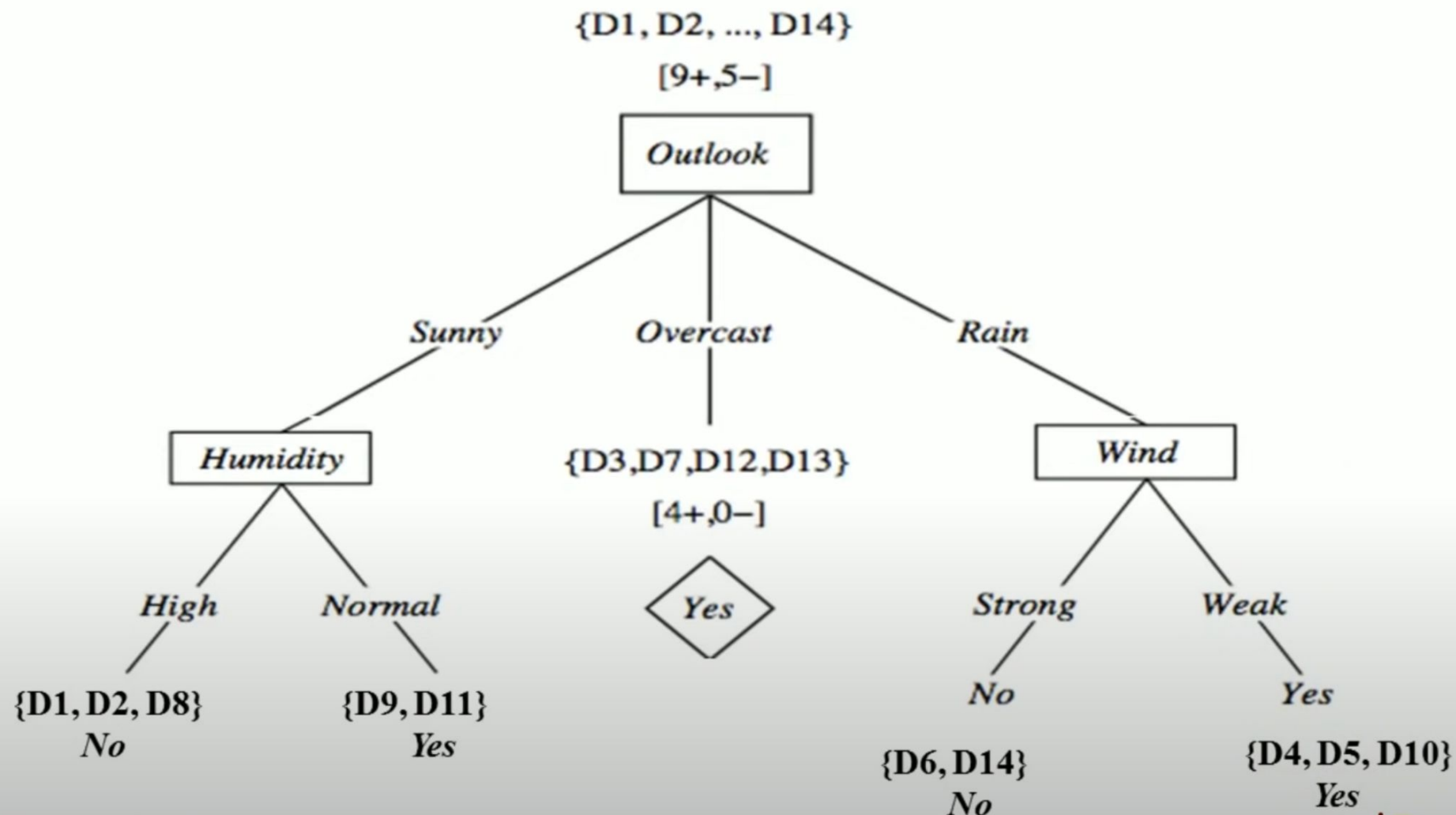
$$Gain(S_{Rain}, Wind) = 0.97 - \left(\frac{2}{5}\right) * 0 - \left(\frac{3}{5}\right) * 0 = 0.97$$

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

$$Gain(S_{Rain}, Temp) = 0.0192$$

$$Gain(S_{Rain}, Humidity) = 0.0192$$

$$Gain(S_{Rain}, Wind) = 0.97$$



Using CART algorithm on same dataset

Step 1: Gini Index of Full Dataset

$$Gini(S) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459$$

Step 1: Dataset Summary

The dataset has 14 instances.

Play Golf	Count
Yes	9
No	5

Gini Index for root:

$$Gini(S) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 1 - 0.41 - 0.13 = \boxed{0.459}$$

Step 2: Try Splits Using Binary CART

CART evaluates **binary splits only** (e.g., Outlook = Sunny? Yes/No).

We'll compute weighted Gini for each binary split.

Split: Outlook = Sunny?

- Yes branch (Sunny): 5 samples (2 Yes, 3 No),

$$Gini = 1 - (2/5)^2 - (3/5)^2 = 0.48$$

- No branch (Not Sunny): 9 samples (7 Yes, 2 No),

$$Gini = 1 - (7/9)^2 - (2/9)^2 \approx 0.346$$

$$Gini_{split} = \frac{5}{14} \cdot 0.48 + \frac{9}{14} \cdot 0.346 \approx 0.398$$

Split: Outlook = Overcast?

- Yes branch (Overcast): 4 samples (4 Yes), Gini = 0
- No branch: 10 samples (5 Yes, 5 No),

$$Gini = 1 - (5/10)^2 - (5/10)^2 = 0.5$$

$$Gini_{split} = \frac{4}{14} \cdot 0 + \frac{10}{14} \cdot 0.5 = 0.357$$

Split: Humidity = High?

- High: 7 samples (3 Yes, 4 No),

$$Gini = 1 - (3/7)^2 - (4/7)^2 \approx 0.489$$

- Not High: 7 samples (6 Yes, 1 No),

$$Gini = 1 - (6/7)^2 - (1/7)^2 \approx 0.245$$

$$Gini_{split} = \frac{7}{14} \cdot 0.489 + \frac{7}{14} \cdot 0.245 = 0.367$$

Split: Windy = True?

- True: 6 samples (3 Yes, 3 No), Gini = 0.5
- False: 8 samples (6 Yes, 2 No),

$$Gini = 1 - (6/8)^2 - (2/8)^2 = 0.375$$

$$Gini_{split} = \frac{6}{14} \cdot 0.5 + \frac{8}{14} \cdot 0.375 = 0.429$$

Best Split: Outlook = Overcast? (Gini = 0.357)

Step 3: First Split - Outlook = Overcast?

Overcast = Yes $\rightarrow$ 4 samples (4 Yes) $\rightarrow$ Pure $\rightarrow$ Yes

Overcast = No $\rightarrow$ 10 samples (5 Yes, 5 No) $\rightarrow$ Needs further splitting

—

Step 4: Further Split on Outlook Overcast

Now use the remaining 10 samples to test other binary splits.

Try **Humidity = High?** $\rightarrow$ Gives next best split for this subset.

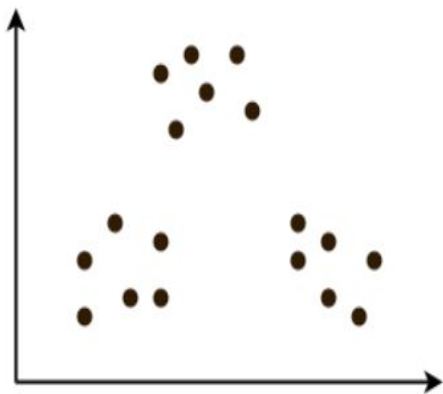
- High: 5 samples (0 Yes, 5 No) $\rightarrow$ Gini = 0
- Not High: 5 samples (5 Yes, 0 No) $\rightarrow$ Gini = 0

$$Gini_{split} = 0 \Rightarrow \text{Perfect split}$$

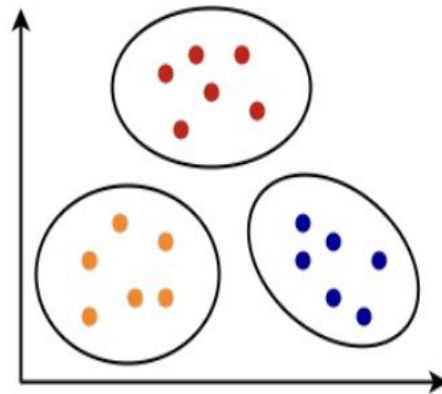
K-means Clustering

K-Means clustering is an unsupervised, iterative clustering technique that partitions a given dataset into k predefined distinct clusters.

A cluster is defined as a collection of data points exhibiting certain similarities.



Before K-Means



After K-Means

Cluster Properties

The clustering is performed such that:

- Each data point belongs to a cluster whose mean (centroid) is nearest to it.
- Data points within the same cluster exhibit a high degree of similarity.
- Data points from different clusters show a high degree of dissimilarity.

K-Means Algorithm

The K-Means clustering algorithm involves the following steps:

Step-1: Choose the number of clusters k .

Step-2: Initialize cluster centers:

- Randomly select k data points as the initial cluster centers.
- Try to select them such that they are as far apart as possible.

Step-3: Assign data points to clusters:

- Calculate the distance between each data point and each cluster center.
- Commonly used distance metric: Euclidean distance.

K-Means Algorithm

Step-4: Cluster assignment:

- Assign each data point to the cluster with the nearest centroid.

Step-5: Recompute centroids:

- For each cluster, compute the new centroid by taking the mean of all data points assigned to that cluster.

Step-6: Repeat:

- Repeat steps 3 to 5 until one of the following conditions is met:
 - Cluster centers do not change.
 - Data point assignments remain unchanged.
 - A maximum number of iterations is reached.

Given

- Number of clusters: $K = 2$
- Initial centroids:
 - $C1 = (185, 72)$
 - $C2 = (170, 56)$
- Data points:

Point	Coordinates
P_1	(185, 72)
P_2	(170, 56)
P_3	(168, 60)
P_4	(179, 68)
P_5	(182, 72)
P_6	(188, 77)

Calculation of distances from each centroid

The Euclidean distance is calculated using:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Point	Coordinates	$d(P, C_1)$	$d(P, C_2)$	Cluster
P_1	(185, 72)	0	$\sqrt{15^2 + 16^2} = 21.93$	C_1
P_2	(170, 56)	21.93	0	C_2
P_3	(168, 60)	$\sqrt{17^2 + 12^2} = 20.80$	$\sqrt{2^2 + 4^2} = 4.47$	C_2
P_4	(179, 68)	$\sqrt{6^2 + 4^2} = 7.21$	$\sqrt{9^2 + 12^2} = 15.00$	C_1
P_5	(182, 72)	3.00	20.00	C_1
P_6	(188, 77)	$\sqrt{3^2 + 5^2} = 5.83$	27.66	C_1

Clusters after Iteration 1:

- C_1 : P_1, P_4, P_5, P_6
- C_2 : P_2, P_3

New Centroids

- For C_1 :

$$\text{Mean} = \left(\frac{185 + 179 + 182 + 188}{4}, \frac{72 + 68 + 72 + 77}{4} \right) = (183.5, 72.25)$$

- For C_2 :

$$\text{Mean} = \left(\frac{170 + 168}{2}, \frac{56 + 60}{2} \right) = (169, 58)$$

Iteration 2: Recalculate Distance

Point	Coordinates	$d(P, C'_1)$	$d(P, C'_2)$	Cluster
P_1	(185, 72)	$\sqrt{1.5^2 + 0.25^2} \approx 1.52$	$\sqrt{16^2 + 14^2} \approx 21.26$	C_1
P_2	(170, 56)	$\sqrt{13.5^2 + 16.25^2} \approx 21.13$	$\sqrt{1^2 + 2^2} \approx 2.24$	C_2
P_3	(168, 60)	$\sqrt{15.5^2 + 12.25^2} \approx 19.71$	$\sqrt{1^2 + 2^2} \approx 2.24$	C_2
P_4	(179, 68)	$\sqrt{4.5^2 + 4.25^2} \approx 6.20$	$\sqrt{10^2 + 10^2} \approx 14.14$	C_1
P_5	(182, 72)	$\sqrt{1.5^2 + 0.25^2} \approx 1.52$	$\sqrt{13^2 + 14^2} \approx 19.10$	C_1
P_6	(188, 77)	$\sqrt{4.5^2 + 4.75^2} \approx 6.55$	$\sqrt{19^2 + 19^2} \approx 26.87$	C_1

Clusters after Iteration 2:

- C_1 : P_1, P_4, P_5, P_6
- C_2 : P_2, P_3

Conclusion

After two iterations, the clusters did not change. Hence, the algorithm has converged.

The final clusters are:

- Cluster 1 (C1): (185, 72), (179, 68), (182, 72), (188, 77)
- Cluster 2 (C2): (170, 56), (168, 60)

Also plot the clusters.

Cluster the following eight 2D points using the K-Means algorithm with $K = 3$ and perform 2 iterations.

Point	Coordinates
A_1	(2, 10)
A_2	(2, 5)
A_3	(8, 4)
A_4	(5, 8)
A_5	(7, 5)
A_6	(6, 4)
A_7	(1, 2)
A_8	(4, 9)

Logistic Regression

Logistic Regression is a supervised machine learning algorithm used primarily for classification tasks.

Unlike Linear Regression, which predicts continuous values, Logistic Regression estimates the probability that a given input belongs to a particular class.

It is especially useful for binary classification problems where the target variable has only two classes:

- Yes / No
- 0 / 1
- True / False

Assumptions

- Independent Observations: Each sample is independent.
- Binary Output (for binary case): Target variable should be 0 or 1.
- Linearity of Logit: Linear relationship between predictors and log-odds.
- No Outliers: Outliers can distort parameter estimation.
- Sufficient Sample Size: Needed for stability and accuracy.

Sigmoid Activation

The sigmoid function transforms any real-valued number into a value between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\lim_{z \rightarrow \infty} \sigma(z) = 1, \quad \lim_{z \rightarrow -\infty} \sigma(z) = 0$$

Decision Rule:

$$\sigma(z) \geq 0.5 \Rightarrow \text{Class 1}; \quad \text{Else Class 0}$$

Example: An organization wants to determine's an employee salary increase based on their performance. A linear regression will help them to decide such problem



Now, what if the organization wants to know whether an employee would get a promotion or not based on their performance?

The linear graph would not be suitable for this case

So, we need to clip the line at 0 and 1 and convert them into sigmoid curve(S-curve).

Based on the threshold value now the question can be easily answered



Algorithm for Logistic regression

1. Gather or load the dataset and determine independent and dependent variables.
2. Find the parameters like intercept and slope value from the dataset using OLS or gradient descent.
3. The sigmoid function transforms the linear combination of inputs into a probability:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$z = \text{intercept} + \text{slope} * X$

4. Define the threshold and compute the sigmoid value for new data
5. Using the probability value compare with the threshold value and find the class for the new data

Applications

Healthcare: Predicting whether a patient has a disease based on various health indicators.

Marketing: Estimating whether a customer will respond to a marketing campaign.

Finance: Assessing credit risk by predicting whether an applicant will default on a loan.

Advantages

Simplicity: Logistic regression is easy to implement and interpret.

Efficiency: It works well with large datasets and provides quick predictions.

Probabilistic Output: It gives probabilities that can be useful for decision-making.

Disadvantages

Linearity Assumption: Logistic regression assumes a linear relationship between the independent variables and the log odds of the dependent variable.

Not Suitable for Non-linear Problems: When the relationship between variables is non-linear, logistic regression may not perform well without transformation or additional techniques.

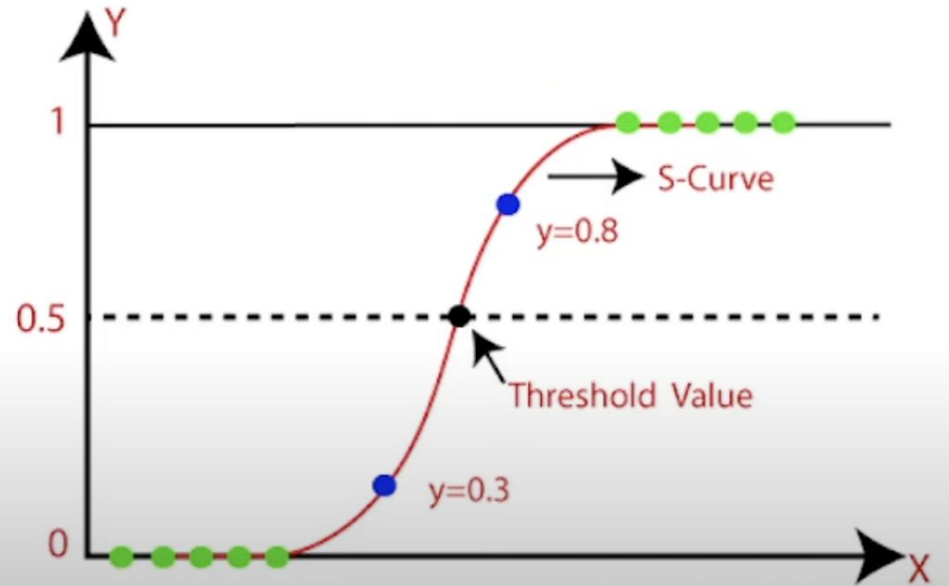
Example

Study (in hours)	Result
29	0
15	0
33	1
28	1
39	1

Calculate the probability of pass for student who studied 34 hours.

Setting 0.5 as the threshold value

- $s(x) = \frac{1}{1+e^{-x}}$



Determine the slope and intercept of the data

Assuming the value of slope and intercept for calculation.

Note: You can use either OLS or gradient descent to find the slope and intercept value.

Slope = 2 (let)

Intercept = -64 (let)

Calculate the probability of the pass for student who studied 33 hours

$$p = \frac{1}{1 + e^{-z}}$$

$$z = \text{slope} * \text{hours} + \text{intercept} = 2 \times 33 - 64 = 2$$

$$p = \frac{1}{1 + e^{-2}} = 0.88$$

if student studies for 33 hours, there is 88% chance he will pass the exam

At least how many hours student should study that makes he will pass the course with the probability of more than 95%.

$$p = \frac{1}{1 + e^{-z}} = 0.95$$

$$0.95 * (1 + e^{-z}) = 1$$

$$e^{-z} = 0.0526$$

$$\ln(e^{-z}) = \ln(0.0526)$$

$$z = 2.94$$

$$z = 2.94$$

$$z = -64 + 2 * \text{hours}$$

$$2.94 = -64 + 2 * \text{hours}$$

$$\text{hours} = 33.47 \text{hours}$$

The student should study at least 33.47 hours
to ensure he will pass the exam with more than 95% probability

Importing Package

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, classification_report,  
confusion_matrix
```

```
import seaborn as sns
```

```
study_hours = np.array([29, 15, 33, 28, 39]).reshape(-1, 1)
```

```
results = np.array([0, 0, 1, 1, 1])
```

Model Evaluation

```
print("Dataset:")  
print("Study Hours | Result")  
print("-" * 20)  
for i in range(len(study_hours)):  
    print(f"{study_hours[i][0]:10d} | {results[i]:6d}")  
  
# Create and fit logistic regression model  
model = LogisticRegression()  
model.fit(study_hours, results)
```

Predictions

```
# Model parameters
```

```
print(f"\nModel Parameters:")
```

```
print(f"Coefficient (slope): {model.coef_[0][0]:.4f}")
```

```
print(f"Intercept: {model.intercept_[0]:.4f}")
```

```
# Predictions
```

```
predictions = model.predict(study_hours)
```

```
probabilities = model.predict_proba(study_hours)
```

```
print(f"\nPredictions:")

print("Study Hours | Actual | Predicted | Probability of Success")

print("-" * 55)

for i in range(len(study_hours)):

    print(f"{study_hours[i][0]:10d} | {results[i]:6d} | {predictions[i]:9d} |  
{probabilities[i][1]:.4f}")

# Model evaluation

accuracy = accuracy_score(results, predictions)

print(f"\nModel Accuracy: {accuracy:.2%}")

# Confusion Matrix

cm = confusion_matrix(results, predictions)

print(f"\nConfusion Matrix:")

print(cm)
```

```
plt.figure(figsize=(12, 8))  
plt.subplot(1, 2, 1)  
x_range = np.linspace(10, 45, 100).reshape(-1, 1)  
y_prob = model.predict_proba(x_range)[:, 1]  
plt.scatter(study_hours, results, color='red', s=100, label='Actual Data', zorder=5)  
plt.plot(x_range, y_prob, 'b-', linewidth=2, label='Logistic Curve')  
plt.axhline(y=0.5, color='gray', linestyle='--', alpha=0.7, label='Decision Boundary (0.5)')  
plt.xlabel('Study Hours')  
plt.ylabel('Probability of Success')  
plt.title('Logistic Regression: Study Hours vs Success Probability')  
plt.legend()  
plt.grid(True, alpha=0.3)
```

```
# Plot 2: Confusion Matrix Heatmap
plt.subplot(1, 2, 2)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Fail (0)', 'Pass (1)'],
            yticklabels=['Fail (0)', 'Pass (1)'])
plt.title('Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.tight_layout()
plt.show()
```

```
print(f"\nLogistic Regression Equation:")
```

```
print(f"P(success) = 1 / (1 + e^(-({model.coef_[0][0]:.4f} * study_hours +  
{model.intercept_[0]:.4f}))))")
```

```
print(f"\nInterpretation:")
```

```
print(f"- For every additional hour of study, the odds of success increase  
by a factor of {np.exp(model.coef_[0][0]):.3f}")
```

```
print(f"- At {-model.intercept_[0]/model.coef_[0][0]:.1f} study hours, the  
probability of success is 50%")
```

```
# Predict for new values
```

```
new_hours = np.array([20, 25, 30, 35, 40]).reshape(-1, 1)
```

```
new_probs = model.predict_proba(new_hours)[:, 1]
```

```
new_preds = model.predict(new_hours)
```

```
print(f"\nPredictions for new study hours:")
```

```
print("Hours | Probability | Prediction")
```

```
print("-" * 30)
```

```
for i in range(len(new_hours)):
```

```
    print(f"{new_hours[i][0]:5d} | {new_probs[i]:11.3f} | {new_preds[i]:10d}")
```